Linux Fundamentals (Module 6)

Presented by

Dr. Priyanka Chakraborty

Shell variables

- Shell supports variables that are useful both in the command line and shell scripts
- A variable assignment is of the form variable=value (no space around=) but its evaluation requires the $ as prefix to the variable name

- **$ count=5** ---- no $ is required for assignment but needed for evaluation
- **$ echo $count** ---- output will be 5

A variable can also be assigned the value of another variable.

$ total=$count ---- assigning a value to another variable

$ echo $total ---- output will be 5

Shell variables

- **Where to use the shell variable?**

Setting pathnames– If a pathname is used several times in a script, we should assign it to a variable.

We can then use it as an argument to any command. Let's use it with **cd** command

$ progs='/home/kumar/c_progs'

$ cd $progs ; pwd **-----**output will be /home/kumar/c_progs

Shell Programming

- The syntax for Variables in Shell Scripting is given below:

- VARIABLE_NAME=VALUE
- Bash
- The rules for Variables in Shell Scripting are as follows:
- Use the equal sign (=) to assign values to variable names.
- Variable names are case sensitive i.e. ‘A’ and ‘a’ are different.
- To refer to a variable use the dollar sign ($) e. $VARIABLE_NAME.
- While updating/changing the variable values use only the variable name with the assignment operator(=) i.e. VARIABLE_NAME= NEW_VALUE.
- No need to define variable type while declaring variables.
- Enclose multiple words or string values within Single Quote (‘ ‘) to consider all characters as input.

3. Read User Input With Prompt Message

The read command used with option -p allows you to prompt a message along with taking user input. You can use echo $(VARIABLE_NAME) to display the user input on the screen. Here’s an example:

Read: Making Scripts interactive

1) #!/bin/bash

- echo "Hello World“

2. Defining Variables

2) #!/bin/bash

name="Tom"

age=12

- echo "$name's age is $age“

3) #!/bin/sh

echo "What is your name?"

read PERSON

echo "Hello, $PERSON"

4) **Arithmetic Operations**

Perform basic arithmetic operations.

#!/bin/bash

num1=10

num2=20

sum=$((num1 + num2))

echo "The sum is: $sum“

Shell Programming

4) #!/bin/bash

```
echo "enter two numbers"
read num1 num2    //read can take multiple arguments
num3=$(($num1+$num2))
num4=$(($num1-$num2))
num5=$(($num1/num2))
num6=$(($num1%num2))
num7=$(($num1*num2))
echo "Sum is $num3"
echo "Subtraction:$num4"
echo "Division : $num5"
echo "Module : $num6"
echo "Multiplication: $num7"
```

OUTPUT:

```
enter two numbers
8 4
Sum is 12
Subtraction:4:
Division : 2
Module : 0
Multiplication: 32
```

Shell Programming: Bitwise Operation

```
5) #!/bin/bash
echo "enter two numbers“
read num1num2
num3=$((num1&num2))
num4=$((num1|num2))
num5=$((num1^num2))
echo “AND is $num3”
echo “OR:$num4”
echo “NOT : $num5”
```

Outputs:

```
enter two numbers
5 6
AND is 4
OR:7
NOT : 3
```

Shell Programming

13. Performing Mathematical Operations Without Storing

To perform multiple mathematical operations without storing them in a separate variable, write the operations inside echo . This is how:

```
#!/bin/bash
read -p "Enter a number:" num1
read -p "Enter a smaller number:" num2
echo "Addition: $(($num1 + $num2))"
echo "Subtraction: $(($num1 - $num2))"
echo "Multiplication: $(($num1 * $num2))"
echo "Division: $(($num1 / $num2))"
```

Bash

Output:

Enter a number:35

Enter a smaller number:15

Addition: 50 Subtraction: 20

Multiplication: 525 Division: 2

Conditional Statements

There are total 5 conditional statements which can be used in

bash programming

1) if statement

2) if-else statement

3) if..elif..else..fi statement (Else If ladder)

4) if..then..else..if..then..fi..fi..(Nested if)

5) switch statement

1)if statement

This block will process if specified condition is true.

Syntax:

if [expression]

then

Statement

fi

Example 1:

(a) Implementing if statement

```
#Initializing two variables
a=10
b=20
#Check whether they are equal
if [ $a == $b ]
then
   echo "a is equal to b"
fi
```

(b) #Check whether they are not equal

```
if [ $a != $b ]
then
   echo "a is not equal to b"
fi
```

Output

```
$bash -f main.sh
a is not equal to b
```

Conditional Statements (if else statement)

1. -f FILE: Check if a file exists

```
if [ -f "testfile.txt" ]
then
   echo "The file exists."
else
   echo "The file does not exist."
fi
```

2. -d DIR: Checks if a directory exists

This example checks if a directory /home/user exists.

```
if [ -d "/home/user" ]
then
   echo "Directory exists."
else
   echo "Directory does not exist."
fi
```

3. -z STRING: Tests if a string is empty

This example prompts the user to enter a string and checks if it is empty.

```
#!/bin/bash
echo "Enter a string: "
read string
if [ -z "$string" ]
then
   echo "The string is empty."
else
   echo "The string is not empty."
fi
```

```
echo "enter numbers“
read a b
if [ $a = $b ]
Then
echo "numbers equal“
Else
echo "not equal“
fi
```

Numerical Comparison Operators:

(i) - eq equal to

(ii) – ne not equal to

(iii) **–gt** greater than

(iv) **–ge** greater than or equal to

(v) **–lt** less than

(vi) **–le** less than or equal to

Conditional Statements (if else statement)

4. STRING1=STRING2: Tests if two strings are equal

```
echo "Enter string1:"
read string1
echo "Enter string2:"
read string2
if [ "$string1" = "$string2" ]
then
    echo "The strings are equal."
else
    echo "The strings are not equal."
fi
```

5. NUM1 -eq NUM2: Tests if two numbers are equal or not

```
echo "Enter 1st number:"
read num1
echo "Enter 2nd number:"
read num2
if [ "$num1" -eq "$num2" ]
then
    echo "The numbers are equal."
else
    echo "The numbers are not equal."
fi
```

** -eq is used for integer string like “1111”

** = is used for character string like “abcd”

Shell Programming (if else statement)

Example 2:

Implementing if-else statement

```
#!/bin/bash
echo "enter 2 variables“
read a f
if [ $a == $f ]
then
        echo "equal“
else
        echo " not equal“
fi
```

Output

```
enter 2 variables
55 55
equal
```

3. If-elif-else Condition

Check if a number is positive or negative.

```
#!/bin/bash
echo "enter a number“
read n
if [ $n -eq 0 ]
then
        echo "$n is zero“
elif [ $n -lt 0 ]
then
        echo "$n is negative“
else
        echo "$n is positive“
fi
```

OUTPUT

```
enter a number
-66
-66 is negative
```

Creating scripts that check file existence and permissions

Checking file existence and permissions is a common task in shell scripting. You can use if-else statements to evaluate these conditions and take appropriate actions.

Here’s an example of a script that checks if a file exists and has read permissions

```
#!/bin/bash
file_path="/path/to/your/file.txt"
if [ -f "$file_path" ]; then
  if [ -r "$file_path" ]; then
    echo "The file exists and is readable."
  else
    echo "The file exists but is not readable."
  fi
else
  echo "The file does not exist."
fi
```

In this example, the script first checks if the file exists using the -f test. If the file exists, it then checks if the file has read permissions using the -r test. If both conditions are true, it prints “The file exists and is readable.” If the file exists but does not have read permissions, it prints “The file exists but is not readable.” If the file does not exist, it prints “The file does not exist.”

Question: Write a shell script which will print the maximum of three Numbers taking the user inputs.

```
#!/bin/bash
# Prompt the user for three numbers
echo "Enter the first number:"
read num1
echo "Enter the second number:"
read num2
echo "Enter the third number:"
read num3
# Compare and find the largest
if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ]; then
    max=$num1
elif [ $num2 -gt $num1 ] && [ $num2 -gt $num3 ]; then
    max=$num2
else
    max=$num3
fi
```

Question: Write a shell script to calculate sum of the first n natural numbers using the for loop.

```
#!/bin/bash

# Ask user for input
echo "Enter a positive integer:"
read n
sum=0
# Loop from 1 to n and add each number to sum
for (( i=1; i<=n; i++ ))
do
    sum=$((sum + i))
done
# Print the result
echo "Sum of the first $n natural numbers is: $sum"
```

For Loop Examples in Bash Scripting

Example 1: Print Numbers From 5 to 1

```
#!/bin/bash
for ((i=5; i>=1; i--))
do
echo $i
done
```

Example 2: Print Even Numbers From 1 to 10

```
#!/bin/bash
for (( i=1; i<=10; i++ ))
do
if [ $((i%2)) == 0 ]
then
echo $i
fi
done
```

For Loop Examples in Bash Scripting

- Example 3: Print the Multiplication Table of a Number.

```
#!/bin/bash
read -p "Enter a number: " num
for (( i=1; i<=10; i++ ))
do
echo "$num x $i = $((num*i))"
done
```

Output >

```
Enter a number: 12
12 x 1 = 12
12 x 2 = 24
12 x 3 = 36
12 x 4 = 48
12 x 5 = 60
12 x 6 = 72
12 x 7 = 84
12 x 8 = 96
12 x 9 = 108
12 x 10 = 120
```

Example 7: Calculate the Sum of the First “n” Numbers

To calculate the sum of the first n numbers run a for loop and addition operation till n.

Code >

```
#!/bin/bash
read -p "Enter a number: " num
sum=0
for (( i=1; i<=$num; i++ ))
do
sum=$((sum + i))
done
echo “Sum of first $num numbers: $sum”
```

Output

Enter a number: 100

Sum of first 100 numbers: 5050

Shell Programming

15. Check If a Number is an Even or Odd

If a number is even, then its remainder after dividing by 2 will be 0. Otherwise, the number is odd. So, use the remainder operator % within if-else to check if a number is even or odd. Here's how:

```
#!/bin/bash
read -p "Enter a number:" num
if [ $((num%2)) == 0 ]
then
echo "The number is even"
else
echo "The number is odd"
fi
```

Output:

```
Enter a number:25
The number is odd
```

Array in Shell Script

1. Printing All Elements

- ${arr[@]}: This syntax is used to print each element of the array separately. It displays all elements in the array.

•${arr[*]}: Similar to the previous line, this syntax prints all elements of the array as a single string.

2.Printing the First Element:

•echo "The first element:": A message is printed to indicate that the first element of the array is being displayed.

•${arr[0]}: This syntax retrieves and displays the first element of the array. In Bash, array indexing starts at 0.

3.Printing a Selected Index Element:

•selected_index=3: A variable named selected_index is assigned the value 3. This variable represents the desired index in the array.

•echo "Selected index element at index $selected_index:": A message is printed to indicate the selected index.

•${arr[$selected_index]}: Using the value stored in selected_index, this syntax retrieves and displays the element at the specified index

•(index 3 in this case).

Array in Shell Script

4.Printing Elements from a Particular Index:

- echo "Elements from a particular index:": A message is printed to indicate that elements from a specific index are being displayed.
- ${arr[@]:2}: This syntax extracts and displays all elements starting from index 2 in the array. It prints each element separately.
- ${arr[*]:2}: Similar to the previous line, this syntax displays the elements as a single string, starting from index 2.

5.Printing Elements in a Range:

- echo "Elements in a range:": A message is printed to indicate that elements within a specified range are being displayed.
- ${arr[@]:1:3}: This syntax extracts and displays elements starting from index 1 up to index 3 (inclusive). It prints each element separately.
- ${arr[*]:1:3}: Similar to the previous line, this syntax displays the extracted elements as a single string.

Array in Shell Script

```
#!/bin/bash
# To declare a static Array
arr=("Jayesh" "Shivang" "1" "Vipul" "Nishant" "2")

# To print all elements of the array
echo "All elements of the array:"
echo "${arr[@]}"
echo "${arr[*]}"
# To print the first element
echo "The first element:"
echo "${arr[0]}"
# To print a selected index element
selected_index=3
echo "Selected index element at index $selected_index:"
echo "${arr[$selected_index]}"

# To print elements from a particular index
echo "Elements from a particular index:"
echo "${arr[@]:2}"  # Prints elements starting from index 2
echo "${arr[*]:2}"  # Prints elements starting from index 2
```

For Loop Examples in Bash Scripting

Example 5: Loop Through Array Elements

For accessing each array element you can use the for loop in the following manner. Indicate the desired array using "${ARRAY_NAME[@]}" and access each item stored in the array.

```
#!/bin/bash
arr=("mango" "grape" "apple" "cherry" "orange")
for item in "${arr[@]}"; do
echo $item
done
```

Output

```
mango
grape
apple
cherry
Orange
```

```
#!/bin/bash
s=("football" "cricket" "hockey")
for n in ${s[@]};
do
    echo $n
done
```

Shell Programming

8) Create a shell script that checks if a file exists in the current directory.

Here's a script that checks if a file named "example.txt" exists in the current directory:

```
#!/bin/bash

file="example.txt"

# Check if the file exists

if [ -e "$file" ]; then

echo "File exists: $file"

else

echo "File not found: $file"

fi
```